

CE 206: Engineering Computation Sessional

System of Linear Equations

Gauss Elimination

$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{array} \right] \quad \left. \vphantom{\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{array}} \right\} \text{(a) Forward elimination}$$

$$\downarrow$$
$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ & a'_{22} & a'_{23} & b'_2 \\ & & a''_{33} & b''_3 \end{array} \right]$$

$$\downarrow$$
$$x_3 = b''_3 / a''_{33}$$

$$\left. \begin{array}{l} x_2 = (b'_2 - a'_{23}x_3) / a'_{22} \\ x_1 = (b_1 - a_{13}x_3 - a_{12}x_2) / a_{11} \end{array} \right\} \text{(b) Back substitution}$$

Forward elimination

- Starting with the first row, add or subtract multiples of that row to eliminate the first coefficient from the second row and beyond.
- Continue this process with the second row to remove the second coefficient from the third row and beyond.
- Stop when an upper triangular matrix remains.

Back substitution

- Starting with the *last* row, solve for the unknown, then substitute that value into the next highest row.
- Because of the upper-triangular nature of the matrix, each row will contain only one more unknown.

Gauss Elimination code

```
function x = GaussNaive(A,b)
% input: A = coefficient matrix  b = right hand side vector
% output: x = solution vector
[m,n] = size(A);
if m~=n, error('Matrix A must be square'); end
nb = n+1;
Aug = [A b];
% forward elimination
for k = 1:n-1
    for i = k+1:n
        factor = Aug(i,k)/Aug(k,k);
        Aug(i,k:nb) = Aug(i,k:nb)-factor*Aug(k,k:nb);
    end
end
% back substitution
x = zeros(n,1);
x(n) = Aug(n,nb)/Aug(n,n);
for i = n-1:-1:1
    x(i) = (Aug(i,nb)-Aug(i,i+1:n)*x(i+1:n))/Aug(i,i);
end
```

Partial Pivoting

Problems with naïve Gauss elimination: a coefficient along the diagonal is 0 or close to 0

Solution: determine the coefficient with the largest absolute value in the column below the pivot element.

Switch the rows so that the largest element is the pivot element.

Class Exercise:

Implement partial pivoting in your GaussNaive.m function file

Also modify the code in a way so that you can display the matrix after each operation.

Tridiagonal systems

A tridiagonal system is a banded system with a bandwidth of 3:

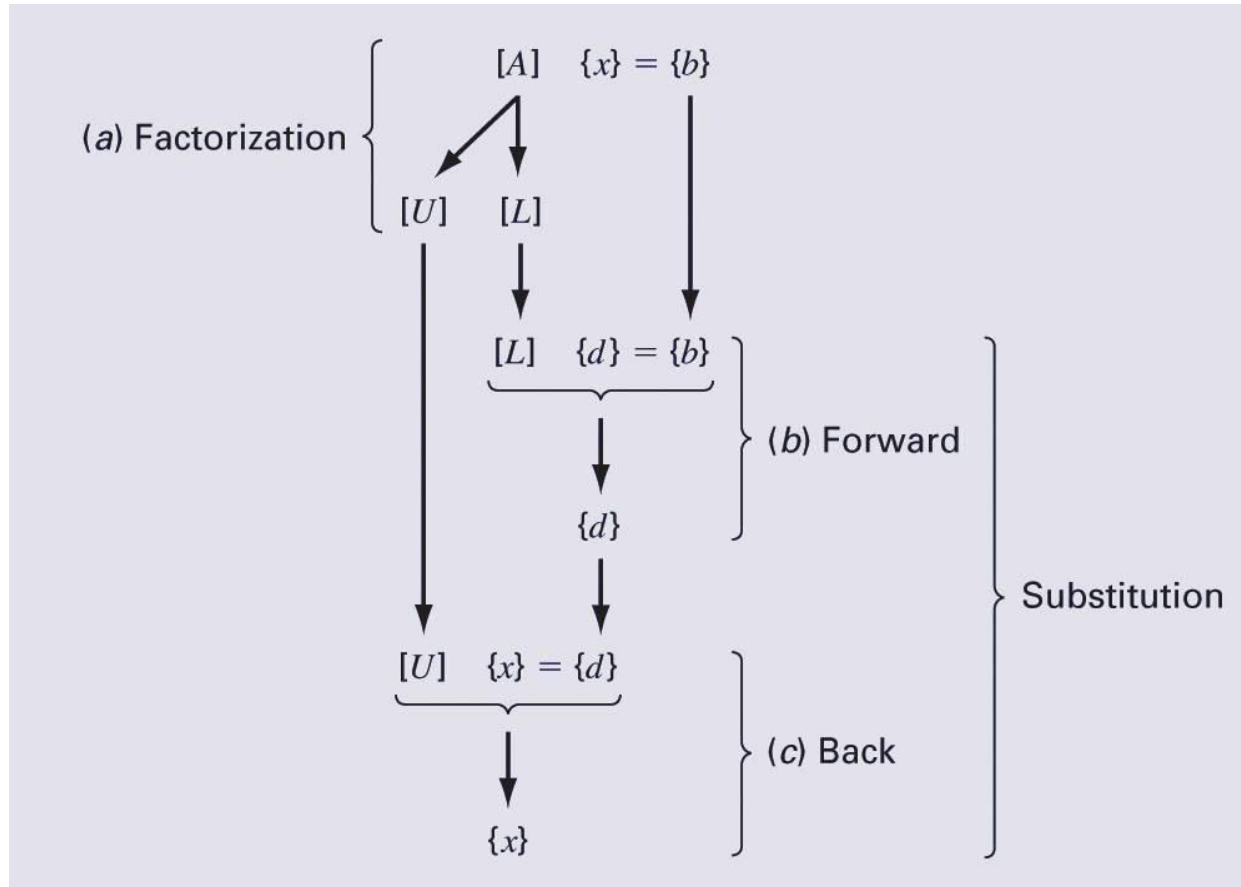
$$\begin{bmatrix} f_1 & g_1 & & & & & & \\ e_2 & f_2 & g_2 & & & & & \\ & e_3 & f_3 & g_3 & & & & \\ & & \cdot & \cdot & \cdot & & & \\ & & & \cdot & \cdot & \cdot & & \\ & & & & \cdot & \cdot & \cdot & \\ & & & & & e_{n-1} & f_{n-1} & g_{n-1} \\ & & & & & & e_n & f_n \end{bmatrix} \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \\ \cdot \\ \cdot \\ \cdot \\ x_{n-1} \\ x_n \end{Bmatrix} = \begin{Bmatrix} r_1 \\ r_2 \\ r_3 \\ \cdot \\ \cdot \\ \cdot \\ r_{n-1} \\ r_n \end{Bmatrix}$$

Tridiagonal systems can be solved using the same method as Gauss elimination, but with much less effort.

Tridiagonal solver

```
function x = Tridiag(e,f,g,r)
% input:
%   e = subdiagonal vector
%   f = diagonal vector
%   g = superdiagonal vector
%   r = right hand side vector
% output:
%   x = solution vector
n=length(f);
% forward elimination
for k = 2:n
    factor = e(k)/f(k-1);
    f(k) = f(k) - factor*g(k-1);
    r(k) = r(k) - factor*r(k-1);
end
% back substitution
x(n) = r(n)/f(n);
for k = n-1:-1:1
    x(k) = (r(k)-g(k)*x(k+1))/f(k);
end
```

LU Factorization



The main advantage is once $[A]$ is decomposed, the same $[L]$ and $[U]$ can be used for multiple $\{b\}$ vectors.

Built-in function `lu()` and `/` operator

To solve $[A]\{x\}=\{b\}$, first decompose $[A]$ to get $[L][U]\{x\}=\{b\}$

$$[\mathbf{L}, \mathbf{U}] = \text{lu}(\mathbf{A})$$

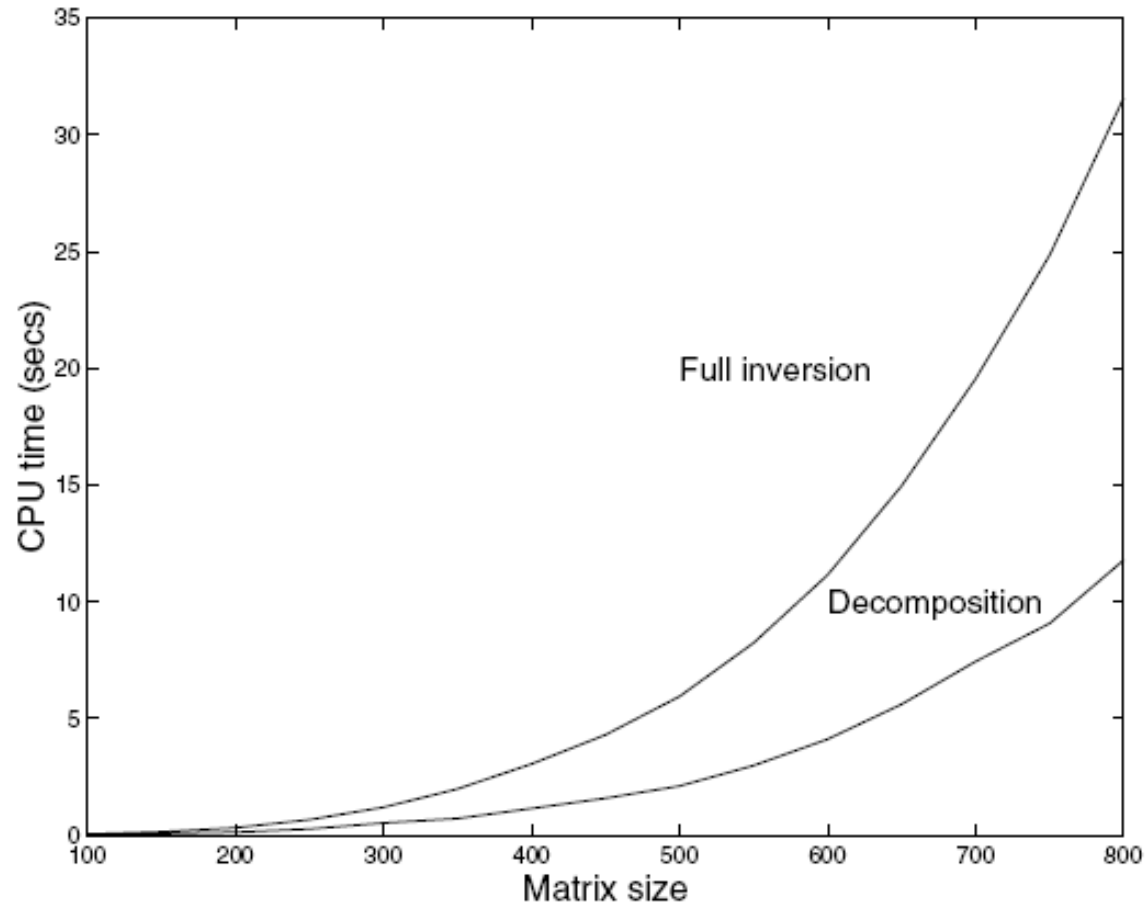
Set up and solve $[L]\{d\}=\{b\}$, where $\{d\}$ can be found using *forward* substitution.

$$\mathbf{d} = \mathbf{L} \backslash \mathbf{b}$$

Set up and solve $[U]\{x\}=\{d\}$, where $\{x\}$ can be found using *backward* substitution.

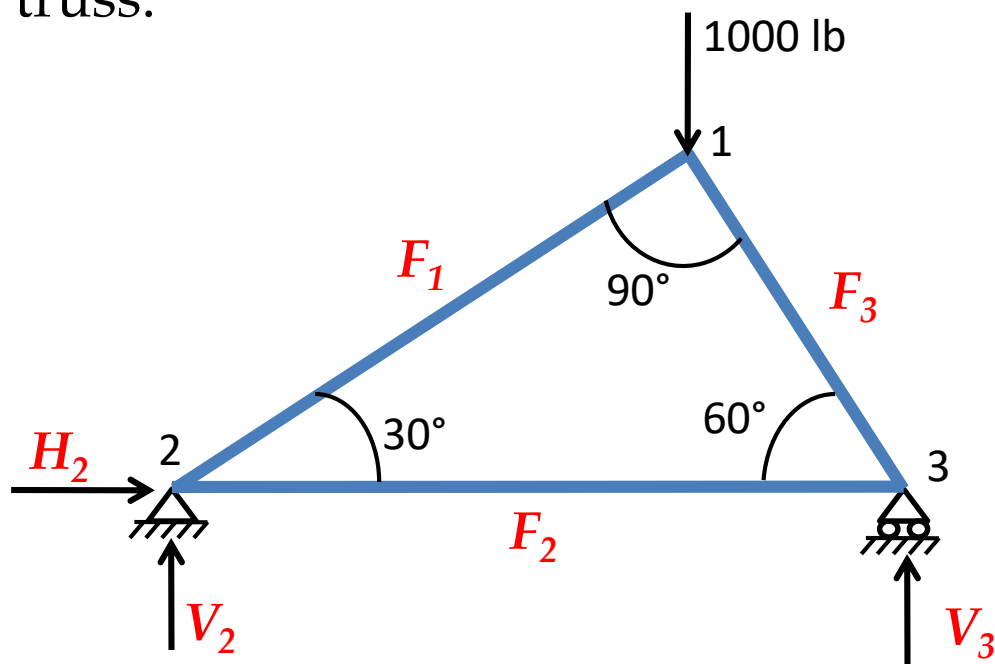
$$\mathbf{x} = \mathbf{U} \backslash \mathbf{d}$$

Decomposition vs. Inverse



Inverting a matrix is computationally expensive

Find out all the bar forces and the external reactions of the following truss:



Node 1:

$$-F_1 \cos 30^\circ + F_3 \cos 60^\circ = 0$$

$$-F_1 \sin 30^\circ - F_3 \sin 60^\circ = 1000$$

Node 2:

$$F_2 + F_1 \cos 30^\circ + H_2 = 0$$

$$F_1 \sin 30^\circ + V_2 = 0$$

Node 3:

$$-F_2 + F_3 \cos 60^\circ = 0$$

$$F_3 \sin 60^\circ + V_3 = 0$$

Exercise 1: Truss Analysis

Unknowns External forcing

$$\begin{bmatrix} 0.866 & 0 & -0.5 & 0 & 0 & 0 \\ 0.5 & 0 & 0.866 & 0 & 0 & 0 \\ -0.866 & -1 & 0 & -1 & 0 & 0 \\ -0.5 & 0 & 0 & 0 & -1 & 0 \\ 0 & 1 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & -0.866 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ H_2 \\ V_2 \\ V_3 \end{bmatrix} = \begin{bmatrix} 0 \\ -1000 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

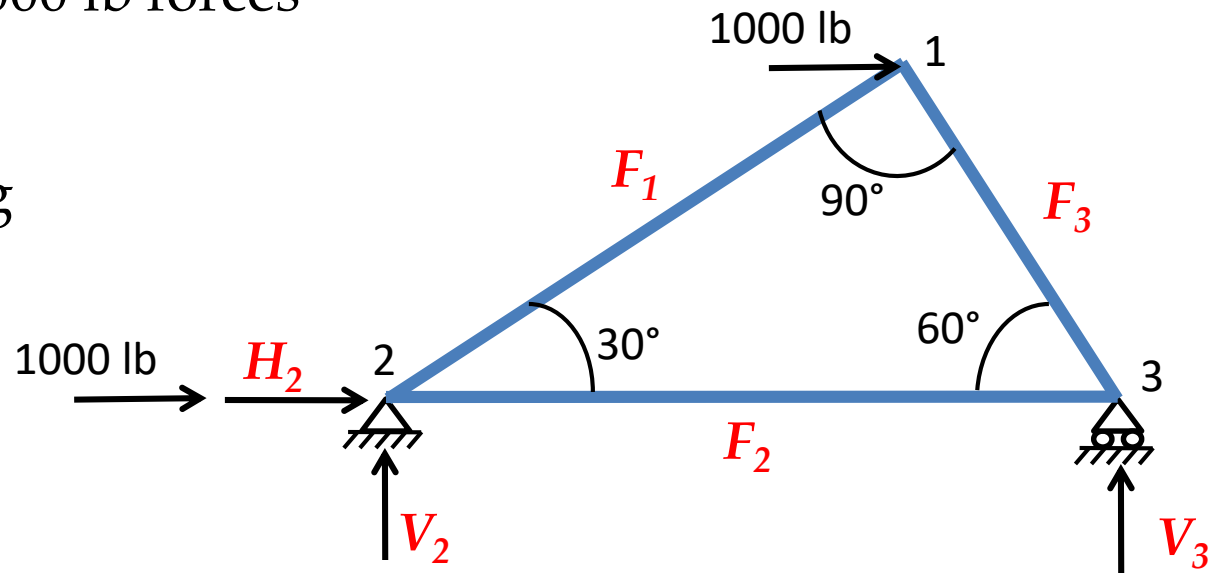
$$\begin{aligned} F_1 &= -500 & F_2 &= 433 & F_3 &= -866 \\ H_2 &= 0 & V_2 &= 250 & V_3 &= 750 \end{aligned}$$

External forces have no effect on LU decomposition, therefore it need not be implemented over and over again for a different external force

Example: Analyze the same truss with two horizontal 1000 lb forces due to wind

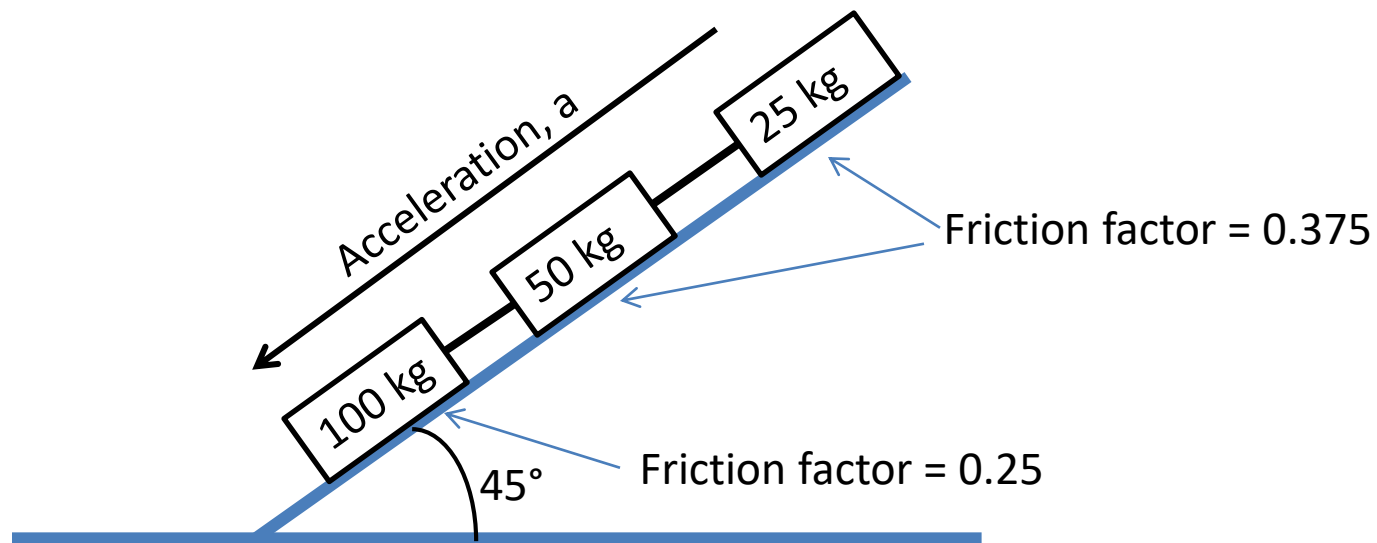
Just change the forcing matrix and solve

$$\begin{bmatrix} -1000 \\ 0 \\ -1000 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$



$$\begin{array}{lll} F_1 = 866 & F_2 = 250 & F_3 = -500 \\ H_2 = -2000 & V_2 = -433 & V_3 = 433 \end{array}$$

Three blocks are connected by a weightless cord and rest on an inclined plane. Develop the set of three simultaneous equations and solve for acceleration and tensions in the cable.



[Use your free body diagram concept from analytic mechanics and apply Newton's second law]

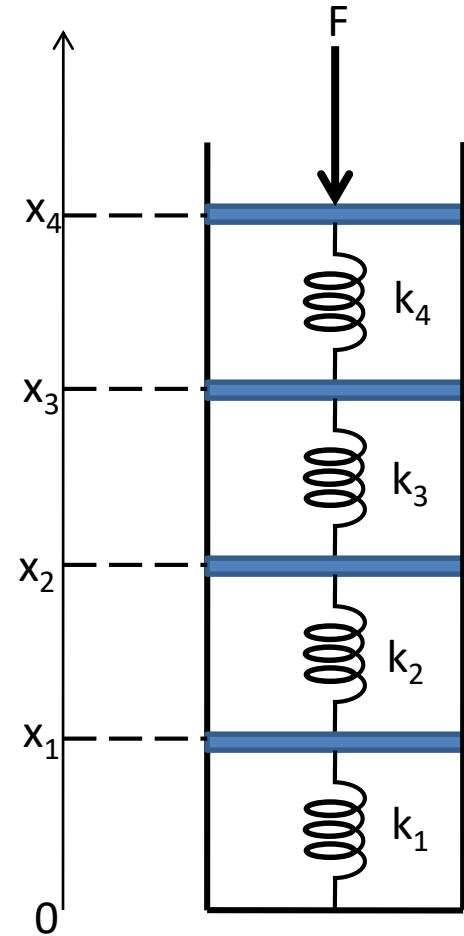
Exercise 3:

A civil engineer involved in construction requires 4800, 5800 and 5700 m³ of sand, fine gravel and coarse gravel respectively, for a building project. There are three pits from which these materials can be obtained. The composition of these pits is

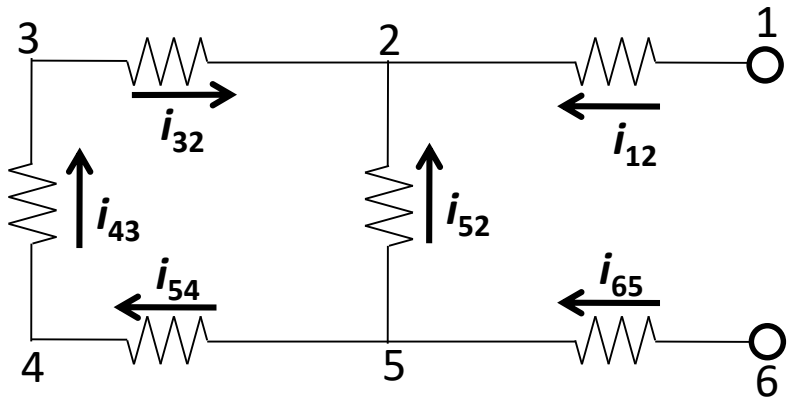
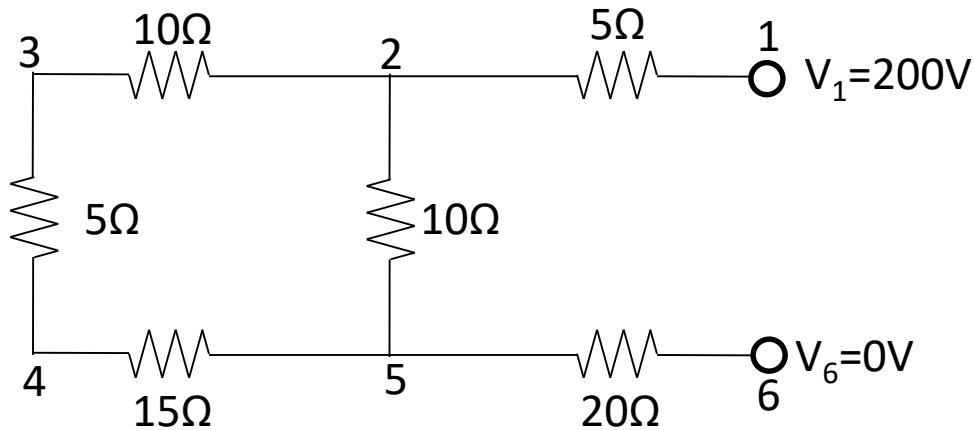
	Sand (%)	Fine Gravel(%)	Coarse gravel (%)
Pit 1	55	30	15
Pit 2	25	45	30
Pit 3	25	20	55

How many cubic meters must be hauled from each pit in order to meet the engineer's needs?

The following figure shows an arrangement of four springs in series being depressed with a force of $F = 1500$ kg. Develop the force balance equations at equilibrium. Solve for the displacements if $k_1 = 100$, $k_2 = 50$, $k_3 = 80$, $k_4 = 200$ N/m.

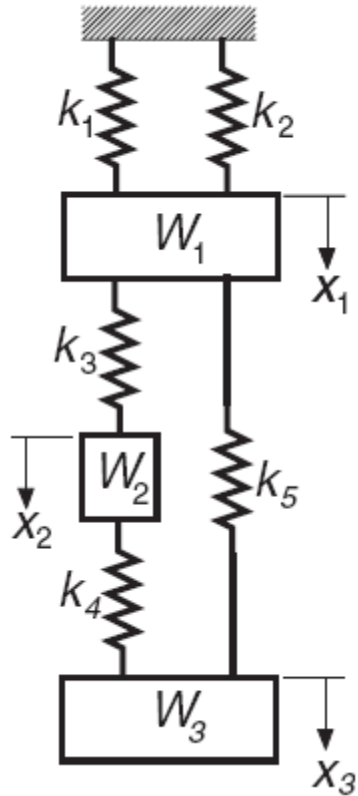


Electrical Circuits



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 10 & -10 & 0 & -15 & -5 \\ 5 & -10 & 0 & -20 & 0 & 0 \end{bmatrix} \begin{bmatrix} i_{12} \\ i_{52} \\ i_{32} \\ i_{65} \\ i_{54} \\ i_{43} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 200 \end{bmatrix}$$

Mass-spring system (Steady-state)



$$\begin{bmatrix} k_1 + k_2 + k_3 + k_5 & -k_3 & -k_5 \\ -k_3 & k_3 + k_4 & -k_4 \\ -k_5 & -k_4 & k_4 + k_5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} W_1 \\ W_2 \\ W_3 \end{bmatrix}$$

Stiffness matrix

The matrix condition number can be used to estimate the precision of solutions of linear algebraic equations.

Cond[A] is obtained by calculating $\text{Cond}[A] = \|A\| \cdot \|A^{-1}\|$

column - sum norm $\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$

Frobenius norm $\|A\|_f = \sqrt{\sum_{i=1}^n \sum_{j=1}^n a_{ij}^2}$

row - sum norm $\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$

spectral norm (2 norm) $\|A\|_2 = (\mu_{\max})^{1/2}$

Matlab built-in function: **cond(A, p)**

Matrix condition number

An ill-conditioned matrix will have very high condition number.

It can be shown that
$$\frac{\|\Delta X\|}{\|X\|} \leq \text{Cond}[A] \frac{\|\Delta A\|}{\|A\|}$$

Relative error of the norm of the computed solution will be very sensitive to the relative error in norm of coeff. of [A]

If the coefficients of [A] are known to t digit precision, the solution [X] may be valid to only $t - \log_{10}(\text{Cond}[A])$ digits.

Example:

$$A = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}$$

$$\text{cond}(A) = 451.2 \gg 1$$